



MobilityAPI

An OGC API – Moving Features server for mobility data, built on MobilityDB

Complete feature walkthrough · canonical AIS example (Danish Maritime Authority)

github.com/MobilityDB/MobilityAPI



From SQL Extension to REST API

- ▶ **MobilityDB** is a PostgreSQL / PostGIS extension for managing temporal and spatio-temporal (mobility) data — vehicles, vessels, pedestrians
- ▶ **MobilityAPI** exposes that data as REST resources for clients that don't speak SQL or the PostgreSQL wire protocol
- ▶ **OGC-compliant** implements the OGC API – Moving Features – Part 1: Core standard (collections, items, temporal geometries, temporal properties)
- ▶ **Built on PyMEOS** for the temporal-data conversion layer between JSON and MobilityDB types
- ▶ **Serves** browser applications, mobile clients, microservices and ETL pipelines

REQUEST FLOW

HTTP client

browser · mobile · ETL



MobilityAPI

REST · OGC API – Moving Features



PyMEOS

temporal-data conversion



MobilityDB / PostgreSQL

tgeompoint · STBOX · TSTZSPAN

MobilityDB SQL vs MobilityAPI REST

PostgreSQL / MobilityDB

```
INSERT INTO temporal_geometries
(feature_id, geometry_type, trajectory,
interpolation)
VALUES (
'219000429', 'MovingPoint',
tgeompoint '[Point(645829 6058110)@2024-08-07
06:00+00,
Point(651280 6058390)@2024-08-07
06:25+00,
Point(656731 6058670)@2024-08-07
06:50+00]',
'Linear'
);
```

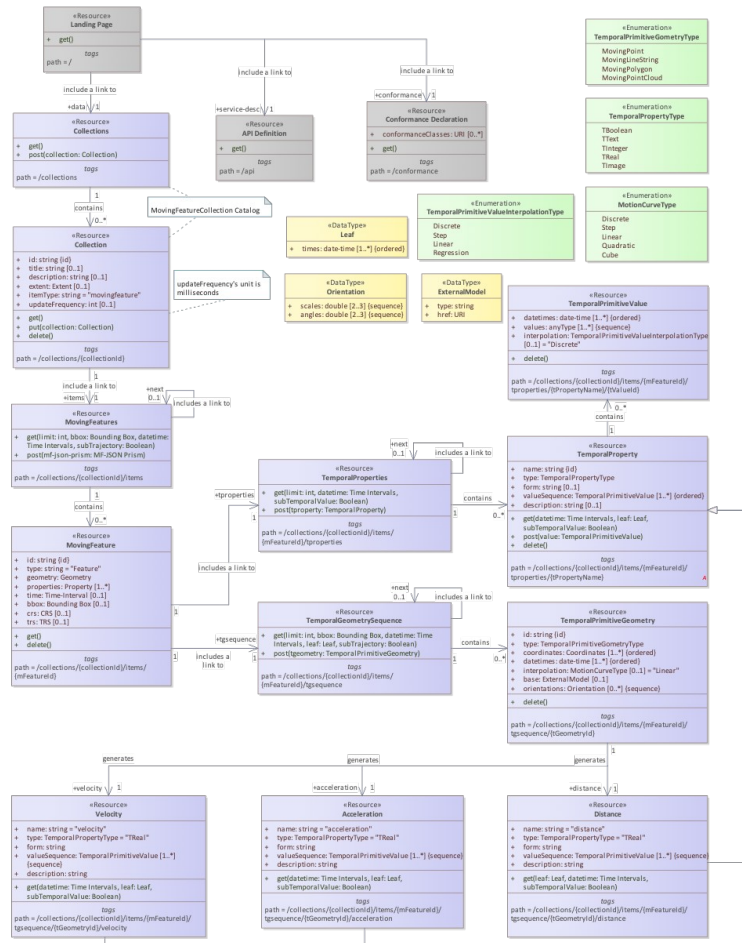
MobilityAPI

```
POST /collections/ships/items
Content-Type: application/json

{
  "type": "Feature",
  "id": "219000429",
  "temporalGeometry": {
    "type": "MovingPoint",
    "datetimes": ["2024-08-07T06:00:00+00",
                  "2024-08-07T06:25:00+00",
                  "2024-08-07T06:50:00+00"],
    "coordinates": [[645829, 6058110],
                    [651280, 6058390],
                    [656731, 6058670]],
    "interpolation": "Linear"
  }
}
```

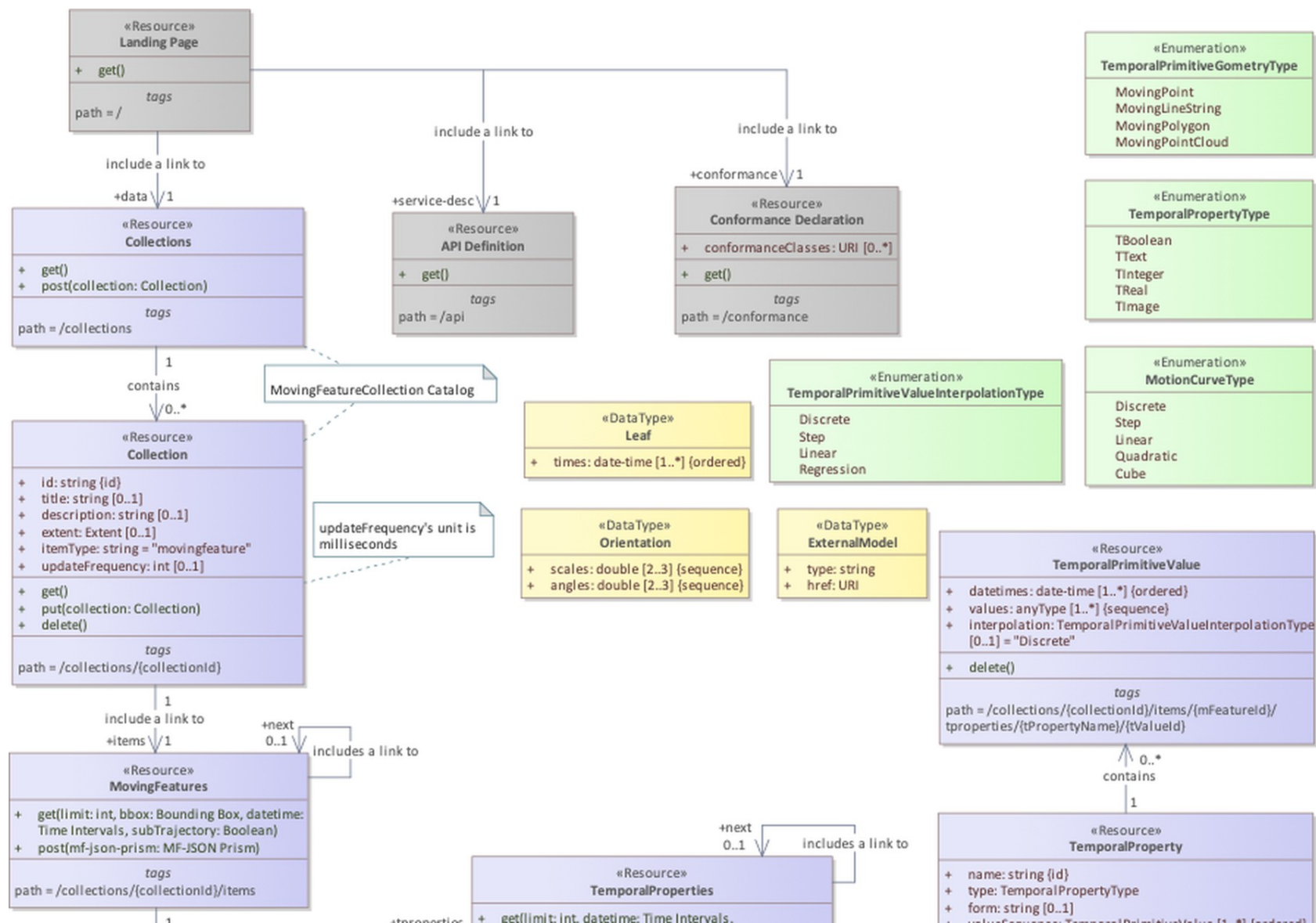
Identical trajectory — the REST client never writes SQL or opens a database connection.

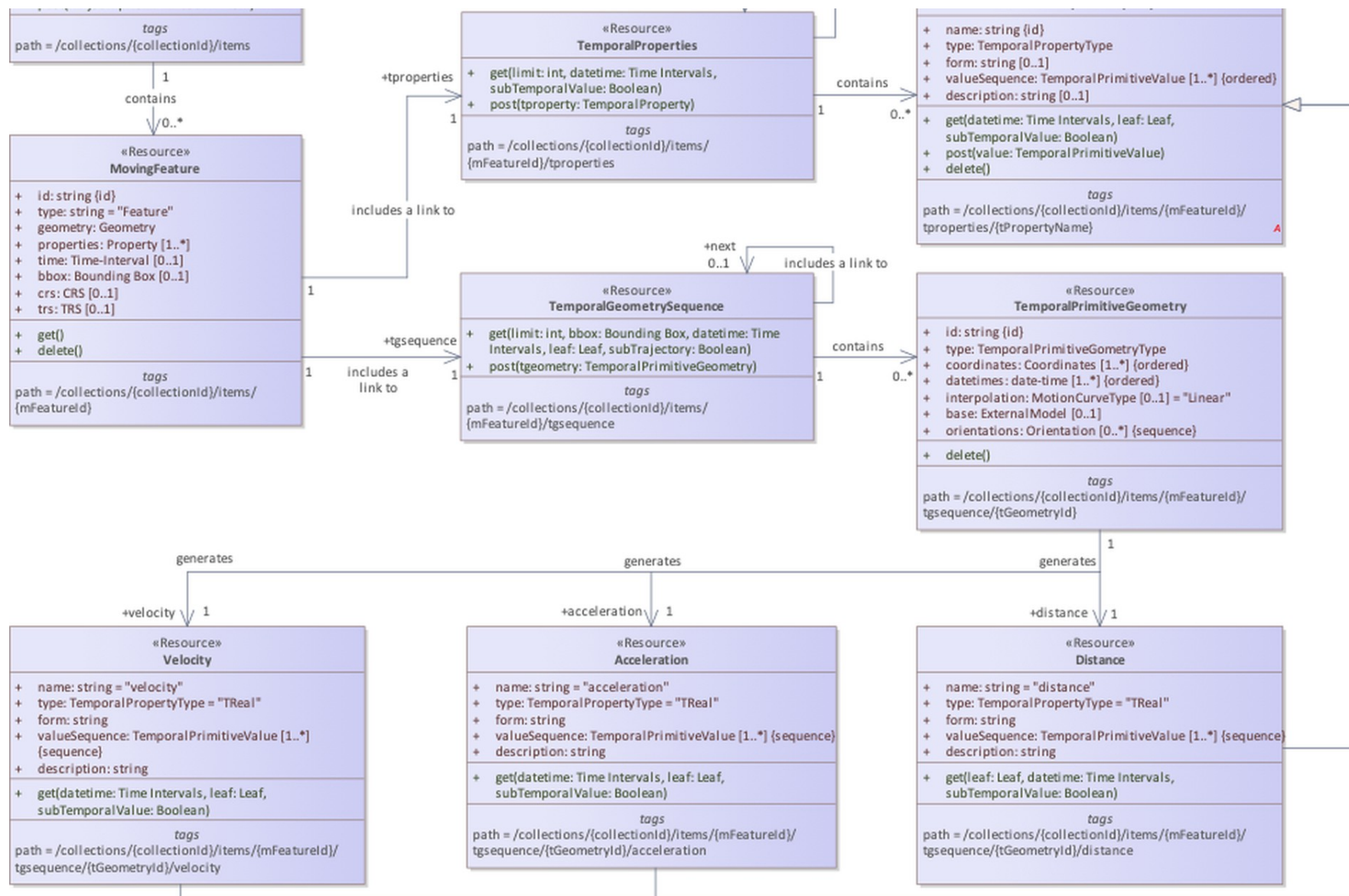
OGC API - Moving Features Data Model



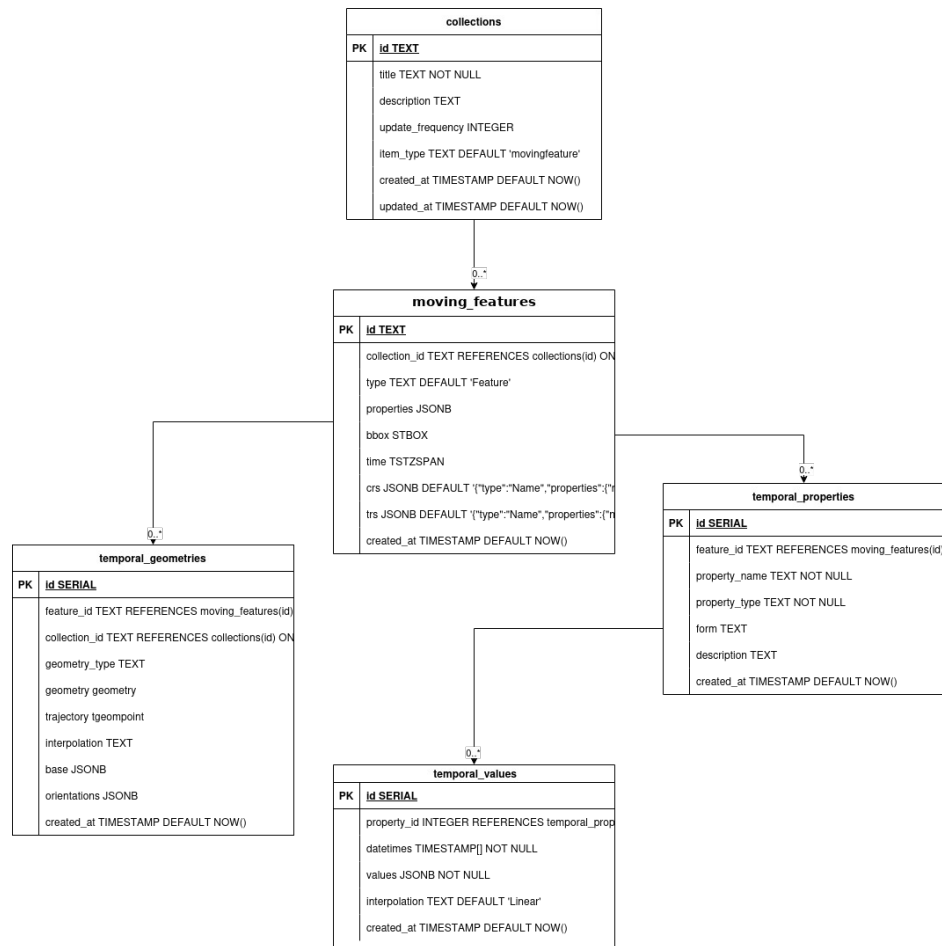
- **Collection** a named group of moving features (itemType movingfeature)
- **MovingFeature** an item with id, properties, crs / trs
- **TemporalGeometry** a MovingPoint — parallel coordinates / datetimes + interpolation
- **TemporalProperties** time-varying attributes attached to a feature
- **Query resources** derived measures over a temporal geometry

Class diagram — OGC API - Moving Features - Part 1: Core (fig. 1)





How MobilityAPI Persists Moving Features



► **collections** metadata: title, description, itemType, updateFrequency

► **moving_features** one row per feature; bbox is an STBOX, time is a TSTZSPAN

► **temporal_geometries** trajectory stored as a MobilityDB tgeopoint

► **temporal_properties** / **temporal_values** time-varying attribute streams

Tables created on first POST /collections — STBOX / TSTZSPAN / tgeopoint are native MobilityDB types.

The Five PostgreSQL Tables in Full

collections

PK id TEXT
 title TEXT
 description TEXT
 update_frequency INTEGER
 item_type TEXT
 created_at TIMESTAMP
 updated_at TIMESTAMP

temporal_geometries

PK id SERIAL
FK feature_id TEXT
FK collection_id TEXT
 geometry_type TEXT
 geometry **geometry**
 trajectory **tgeompoint**
 interpolation TEXT
 base JSONB
 orientations JSONB
 created_at TIMESTAMP

moving_features

PK id TEXT
FK collection_id TEXT
 type TEXT
 properties JSONB
 bbox **STBOX**
 time **TSTZSPAN**
 crs JSONB
 trs JSONB
 created_at TIMESTAMP

temporal_values

PK id SERIAL
FK property_id INTEGER
 datetimes **TIMESTAMP[]**
 values JSONB
 interpolation TEXT
 created_at TIMESTAMP

temporal_properties

PK id SERIAL
FK feature_id TEXT
 property_name TEXT
 property_type TEXT
 form TEXT
 description TEXT
 created_at TIMESTAMP

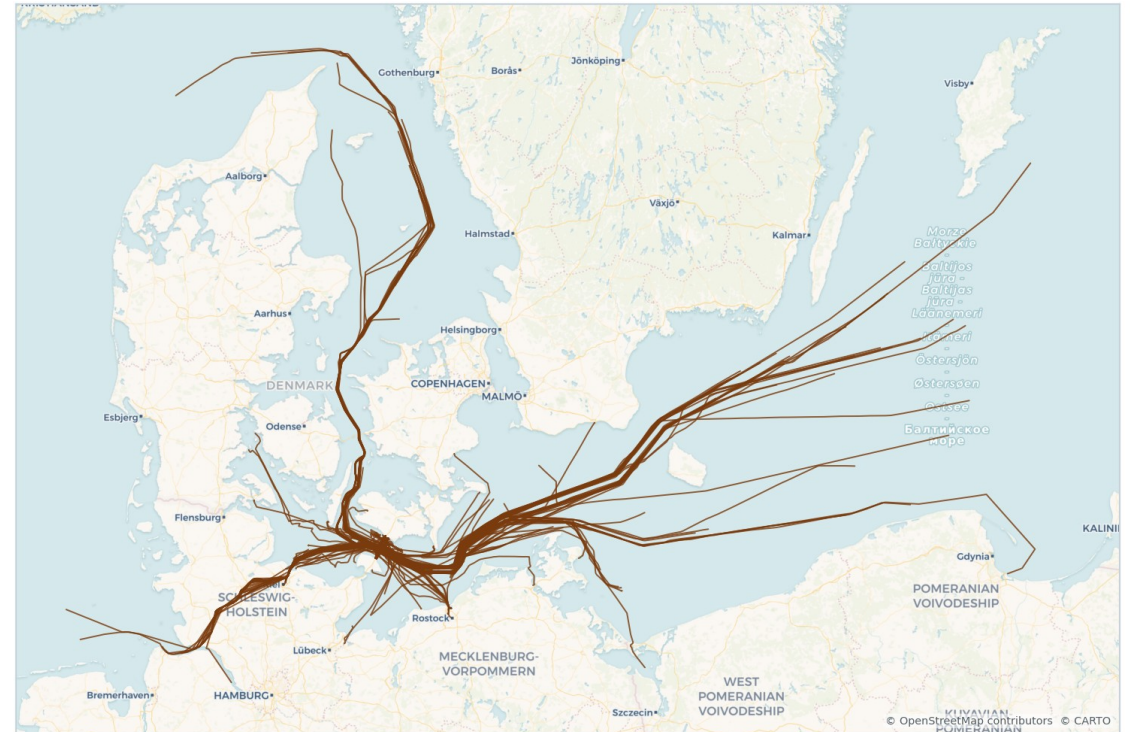
LEGEND

PK primary key FK foreign key
 FK columns reference collections /
 moving_features / temporal_properties
 Highlighted = native MobilityDB /
 PostGIS:

**STBOX · TSTZSPAN · tgeompoint ·
 geometry**

One Day of Danish AIS Ship Traffic

- ▶ **Source** Danish Maritime Authority AIS feed (aisdata.ais.dk)
- ▶ **Window** 2024-08-07 — a single day of observations
- ▶ **143 vessels** each one a MovingPoint trajectory
- ▶ **CRS** EPSG:25832 (ETRS89 / UTM 32N, metres)
- ▶ **Temporal properties** speed (sog), course (cog) and heading per vessel



143 vessel trajectories across Danish and Baltic waters

The Complete Endpoint Surface

COLLECTIONS

GET **POST** /collections

GET **PUT** **DELETE** /collections/{collectionId}

MOVING FEATURES

GET **POST** /collections/{collectionId}/items

GET **DELETE** .../items/{mFeatureId}

TEMPORAL GEOMETRY

GET **POST** .../{mFeatureId}/tgsequence

DELETE .../tgsequence/{tGeometryId}

DERIVED QUERIES

GET .../tgsequence/{tGeometryId}/distance

GET .../{tGeometryId}/velocity · /acceleration

TEMPORAL PROPERTIES

GET **POST** .../{mFeatureId}/tproperties

GET **POST** **DELETE** .../tproperties/{tPropertyName}

DELETE .../tproperties/{tPropertyName}/{tValueId}

WALKTHROUGH

Building the Ships Collection

Create → insert → query → derive → delete, end to end over HTTP

Create a Collection · List Collections

POST /collections

```
{
  "title": "ships",
  "description": "Ship trajectories from the Danish
                 Maritime Authority (one day of AIS).",
  "itemType": "movingfeature",
  "updateFrequency": 1000
}
```

201 Created

```
{
  "id": "ships", "title": "ships",
  "itemType": "movingfeature", "updateFrequency": 1000,
  "extent": null,
  "links": [
    { "href": ".../collections/ships",
      "rel": "self", "type": "application/json" },
    { "href": ".../collections/ships/items",
      "rel": "items", "type": "application/json" }
  ]
}
```

GET /collections**200 OK**

```
{
  "collections": [
    {
      "id": "ships",
      "title": "ships",
      "itemType": "movingfeature",
      "updateFrequency": 1000,
      "links": [ { "rel": "self", ... },
                  { "rel": "items", ... } ]
    }
  ],
  "links": [ { "rel": "self", ... } ]
}
```

Retrieve · Replace · Delete a Collection

GET /collections/ships

```
200 OK
{
  "id": "ships",
  "title": "ships",
  "description": "Ship trajectories ...",
  "itemType": "movingfeature",
  "updateFrequency": 1000,
  "extent": null,
  "links": [ { "rel": "self", ... },
             { "rel": "items", ... } ]
}
```

DELETE /collections/ships

```
200 OK // collection and all its items removed
```

PUT /collections/ships

```
Content-Type: application/json
{
  "title": "ships",
  "description": "Danish AIS – 2024-08-07 (updated)",
  "itemType": "movingfeature",
  "updateFrequency": 2000
}

200 OK
{ "id": "ships", "updateFrequency": 2000, ... }
```

- ▶ PUT replaces the full collection resource
- ▶ DELETE cascades to items, temporal geometries and properties

Insert Vessels as a FeatureCollection

POST /collections/ships/items

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "id": "205106000",
      "properties": { "Name": "MARCUS", "ShipType": "Cargo",
                     "IMO": "9313656", "Destination": "PL GDN" },
      "crs": { "type": "name",
               "properties": { "name":
"urn:ogc:def:crs:EPSG::25832" } },
      "trs": { "type": "Link",
               "properties": { "type": "ogcdef",
                              "href": ".../def/uom/ISO-8601/0/Gregorian" } },
      "temporalGeometry": {
        "type": "MovingPoint",
        "datetimes": ["2024-08-07T11:04:09+00", ...],
        "coordinates": [[521540.85, 5995709.13], ...],
        "interpolation": "Linear"
      }
    }
  ]
}
```

201 Created

```
{
  "message": "Created 1 features",
  "ids": ["205106000"]
}
```

- ▶ Accepts a single Feature or a FeatureCollection
- ▶ MovingFeaturesJSON payload is validated on insert
- ▶ id is the vessel MMSI; properties are free-form
- ▶ Location header returned per created feature

List · Retrieve · Delete Vessels

GET /collections/ships/items?limit=50

```
200 OK
{ "type": "FeatureCollection",
  "features": [
    { "type": "Feature", "id": "205106000",
      "geometry": { "type": "LineString", "coordinates":
[...],
      "properties": { "Name": "MARCUS", "ShipType":
"Cargo" },
      "bbox": [521540.8, 5995709.1, 569588.8, 6024716.7],
      "crs": { ... }, "trs": null,
      "time": ["2024-08-07 11:04:09+00", "2024-08-07
16:43:09+00"],
      "links": [
        { "rel": "self", ... },
        { "rel": ".../temporal-geometry", ... },
        { "rel": ".../temporal-properties", ... } ] }
    ] }
```

GET .../items/{mFeatureId}

DELETE

.../items/
{mFeatureId}

single feature movement (asMFJSON) · remove one vessel



GET .../items returns the FeatureCollection (limit caps page size)

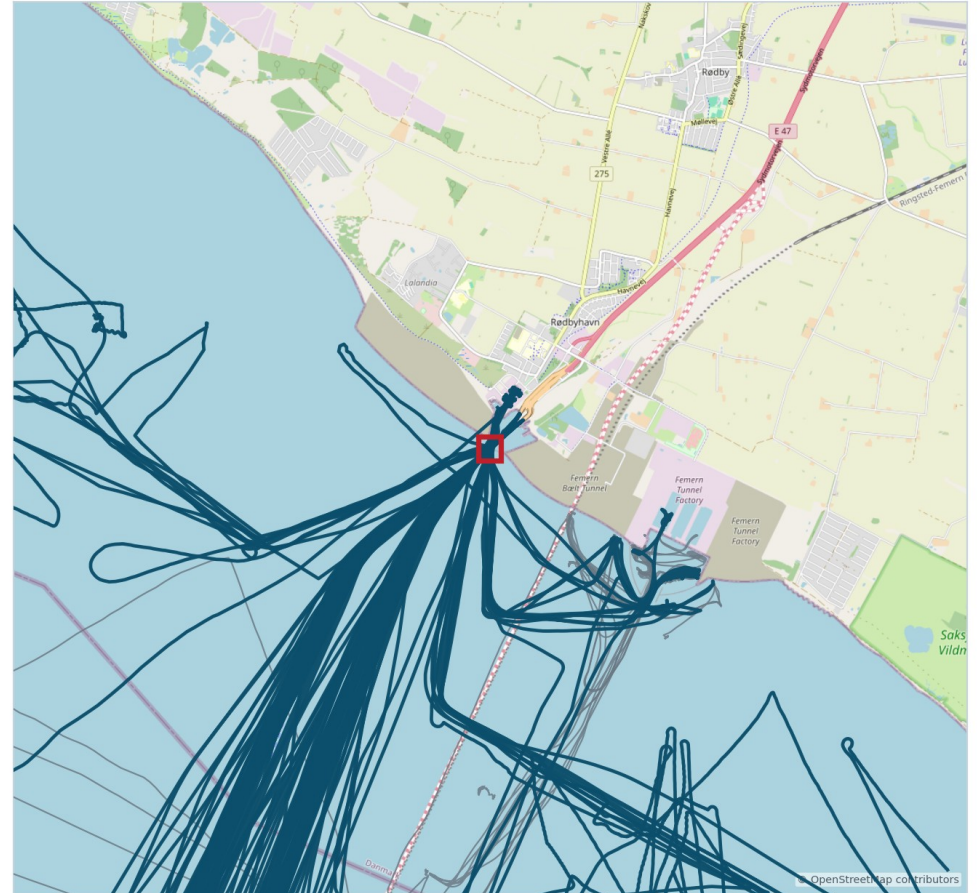
Vessels Intersecting a Bounding Box

GET /collections/ships/items?bbox=...

```
GET /collections/ships/items
  ?bbox=651135,6058230,651422,6058548
  &limit=97

// bbox = minx,miny,maxx,maxy in the storage CRS
// EPSG:25832 (metres) – here a tight envelope
// around Rødby ferry harbour, SE Denmark
```

- ▶ **bbox** keeps vessels whose trajectory intersects the envelope
- ▶ **CRS** coordinates are in the collection's storage CRS (EPSG:25832)
- ▶ **21 of 143** vessels match the Rødby harbour envelope
- ▶ **Composable** with limit, datetime and subTrajectory



Trajectories crossing the Rødby harbour envelope (teal); context in grey

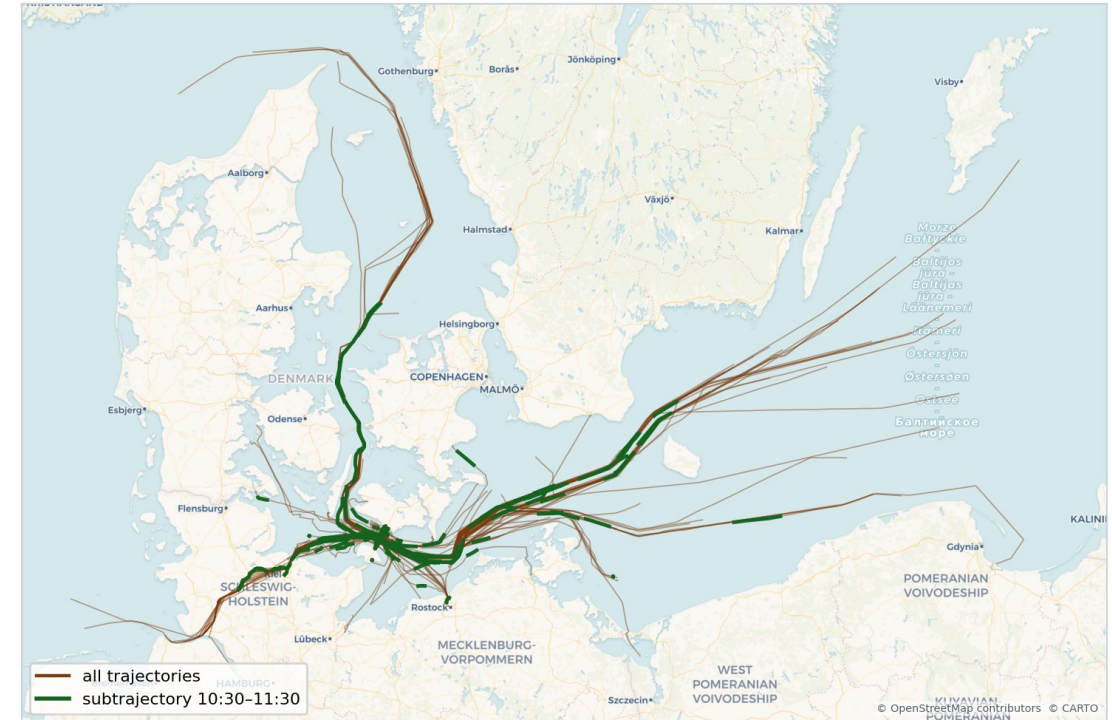
Restrict Trajectories to a Time Interval

GET .../items?subTrajectory=true

```
GET /collections/ships/items
    ?subTrajectory=true
    &datetime=2024-08-07T10:30:00+00
    /2024-08-07T11:30:00+00

// datetime is an RFC 3339 interval (start/end)
// subTrajectory=true clips each vessel to the
// requested window instead of filtering whole trips
```

- ▶ **subTrajectory** clips each trajectory to the datetime interval
- ▶ **datetime** RFC 3339 interval; a bare instant is also accepted
- ▶ **Validation** subTrajectory requires a bounded interval; cannot combine with leaf



Subtrajectories 10:30-11:30 (green) over all trajectories (brown)

The Temporal-Geometry Sequence

GET .../{mFeatureId}/tgsequence

```
200 OK
{
  "type": "TemporalGeometrySequence",
  "geometrySequence": [
    { "id": 360,
      "type": "MovingPoint",
      "datetimes": ["2024-08-07T11:04:09+00", ...],
      "coordinates": [[521540.85, 5995709.13], ...],
      "interpolation": "Linear",
      "base": null }
  ],
  "links": [ { "rel": "self", ... } ],
  "numberMatched": 1, "numberReturned": 1
}
```

Methods on this resource

GET

return the TemporalGeometrySequence for a vessel

POST

append a temporal primitive geometry

DELETE

remove one primitive geometry by tGeometryId

- ▶ Each sequence element carries an id used by the derived-measure queries
- ▶ DELETE targets .../tgsequence/{tGeometryId} — the TemporalPrimitiveGeometry instance

Distance · Velocity · Acceleration

GET .../tgsequence/{tGeometryId}/velocity

200 OK

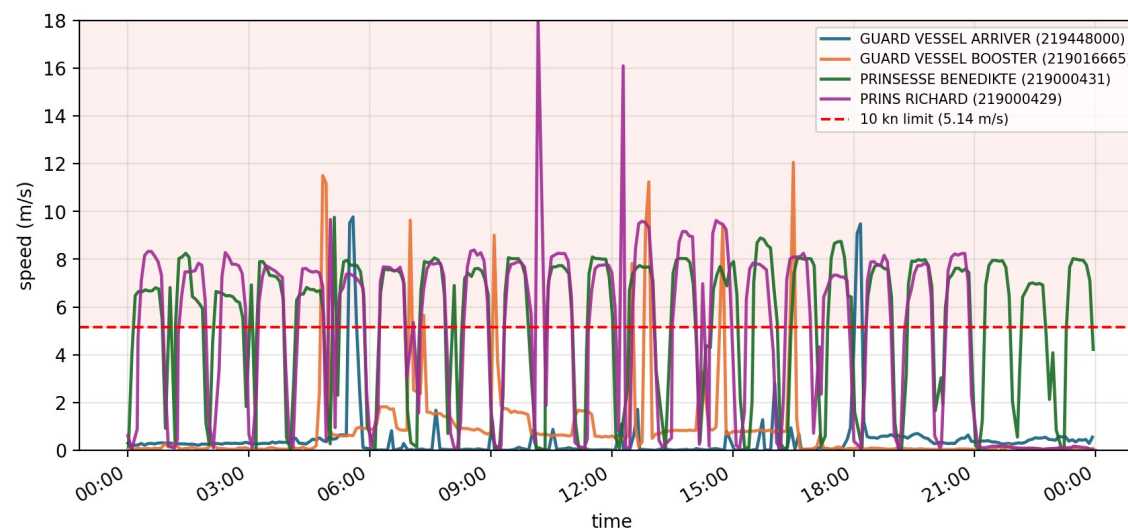
```
{
  "type": "TReal",
  "values": [
    { "time": "2024-08-07T06:00:00+00", "value": 0.79 },
    { "time": "2024-08-07T06:25:00+00", "value": 6.14 },
    { "time": "2024-08-07T06:50:00+00", "value": 0.00 }
  ],
  "unit": "m/s",
  "queryType": "velocity",
  "links": [ { "rel": "self", ... } ]
}
```

distance

velocity

acceleration

interchangeable queryType values — computed by MobilityDB (e.g. speed()).



Velocity profiles of Rødby ferries vs a 10-knot (5.14 m/s) limit

Time-Varying Attributes per Feature

GET .../{mFeatureId}/tproperties

```
200 OK
{
  "temporalProperties": [
    { "name": "sog", "type": "TReal" },
    { "name": "cog", "type": "TReal" },
    { "name": "heading", "type": "TReal" }
  ],
  "links": [ { "rel": "self", ... } ]
}
```

GET .../tproperties/sog

```
{ "name": "sog", "type": "TReal",
  "valueSequence": [
    { "datetimes": ["2024-08-07T11:04:09+00", ...],
      "values": [12.4, 11.8, 0.6, ...] } ] }
```

Methods on this group

GET **POST** /tproperties

list properties · add a new one

GET **POST** /tproperties/{name}

read · append values to one property

DELETE /tproperties/{name}

drop a whole property

DELETE /tproperties/{name}/{tValueId}

drop one primitive value

Property types: TBoolean · TText · TInteger · TReal · TImage

AIS speed (sog), course (cog) and heading ride along as TReal property streams.

Validation, Errors & Hypermedia Links

MovingFeaturesJSON validation

- ▶ Payload validated on every insert
- ▶ Mandatory fields and types checked before persisting
- ▶ Property type restricted to TBoolean / TText / TInteger / TReal / TImage
- ▶ subTrajectory + leaf combinations rejected

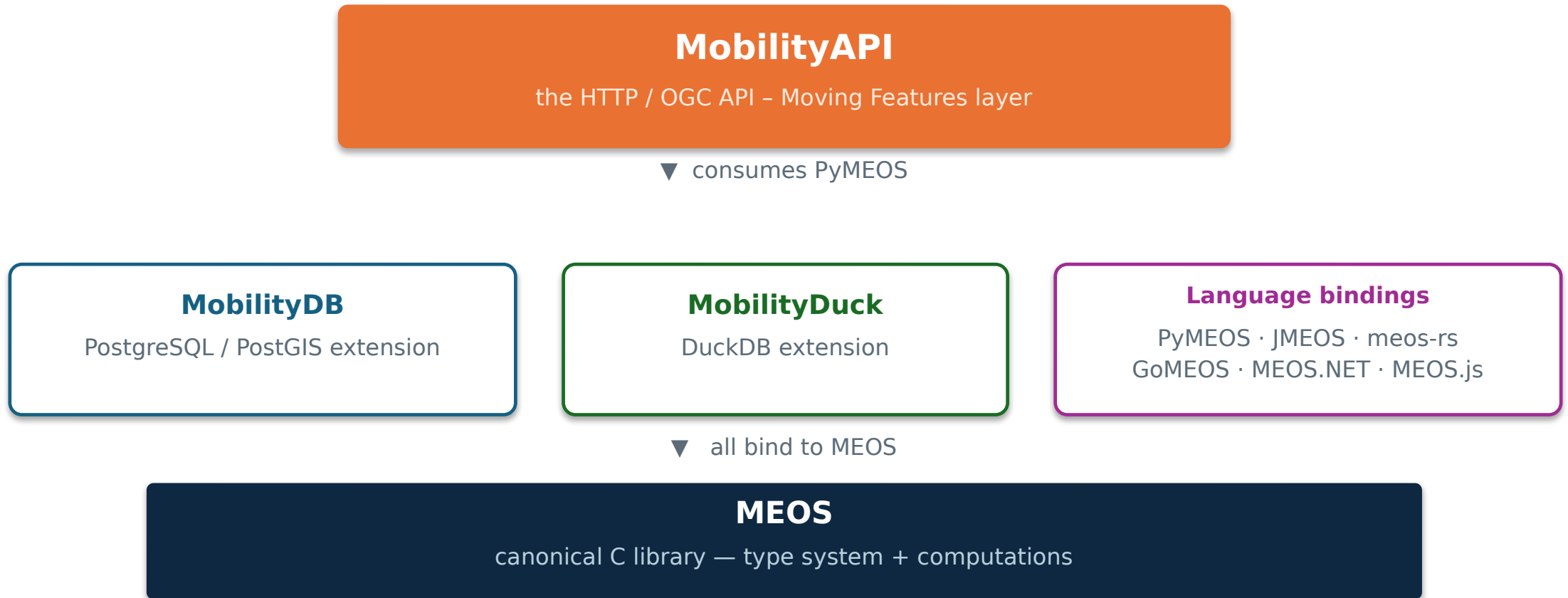
OGC error responses

- ▶ **400** invalid JSON or parameters
- ▶ **404** collection / feature / geometry not found
- ▶ **409** resource already exists
- ▶ **500** internal server error
- ▶ Body: { "code", "description" }

Hypermedia (HATEOAS)

- ▶ Every resource carries links
- ▶ rel: self, items,
- ▶ .../rel/ogc/1.0/temporal-geometry,
- ▶ .../rel/ogc/1.0/temporal-properties
- ▶ Clients navigate without hard-coding paths

Where MobilityAPI Fits in the MEOS Ecosystem



Two Ways to Serve the Same MobilityDB

PyMEOS tier

JSON to type glue in Python; the database does the work

HTTP client

browser · mobile · ETL



Python (PyMEOS)

JSON to temporal-type conversion



MobilityDB (C)

tgeompoint · STBOX · TSTZSPAN

Go tier

thin net/http + pgxpool; no MEOS in the tier (no cgo, no PyMEOS)

HTTP client

browser · mobile · ETL



Go (net/http + pgxpool)

asMFJSON reads · tgeompointFromMFJSON writes



MobilityDB (C)

tgeompoint · STBOX · TSTZSPAN

Both tiers keep every temporal computation in C (the database); only the HTTP / runtime tier differs. The Go tier holds no MEOS: reads stream asMFJSON assembled in SQL, writes call tgeompointFromMFJSON, and a roughly 50-line crs / interpolation adapter is the only application logic, shipped as a single static binary.

Concurrency Ceiling: Same Database, Same SQL

Endpoint	Concurrency	Go req/s	Python req/s
/health (pure tier)	200	85,950	1,337
/collections (light DB)	200	30,709	649
one trajectory	100	2,434	547
ten full trajectories (asMFJSON)	50	90 (100% served)	sheds ~half the load

Same MobilityDB (500 AIS vessel trajectories), identical SQL; the only variable is the HTTP / runtime tier. Load: hey, warm cache, localhost.

► **47 to 64x** faster on Go for tier-bound endpoints (/health, /collections), where the cost is per-request runtime overhead the compiled, no-GIL, goroutine-per-request model absorbs

► **Heavy asMFJSON** raw throughput converges on the database (pool x asMFJSON); the Python tier sheds load under concurrency while Go serves every request at the DB-bound ceiling

► **Shared bottleneck** the database is the limit

Built for Very Large Databases

393 MB

streamed in one export

18 MB

resident memory, bounded by the stream
and not the result size

- ▶ **Streaming responses** the FeatureCollection is written row by row from a server-side cursor, so memory is bounded for any result size
- ▶ **Keyset pagination** next links advance by id with no OFFSET, so paging stays constant cost through arbitrarily large collections
- ▶ **Index-using filters** bbox and datetime push to the MobilityDB GiST index via the overlap operator; subTrajectory clips with atTime
- ▶ **Lakehouse feed** a streaming NDJSON export that DuckDB, MobilityDuck, and Spark ingest directly, then convert to Parquet or Arrow in the lake

Every temporal computation stays in MobilityDB (C); the Go tier is a single static binary that holds no MEOS. A step toward the Data Lake House architecture in the MobilityDB ecosystem.

When to Use Which Tier

PyMEOS / Python tier

- ▶ **Fine for moderate load** and the fastest path to build
- ▶ **PyMEOS in process** is the JSON to temporal-type glue; MobilityDB does the computation
- ▶ **Front-end ceiling** is set by the Python runtime model under high concurrency

Thin compiled tier (Go)

- ▶ **Scalable choice** under high concurrency: a thin Go tier over MobilityDB SQL
- ▶ **Fat DB, thin API** in the pg_featureserv / PostgREST tradition: the database holds the logic, the tier just serves
- ▶ **Single static binary** no cgo, no MEOS in the tier; reads use asMFJSON, writes use tgeompointFromMFJSON

The comparison used a controlled threaded-Python baseline. A FastAPI / uvicorn or multi-worker deployment would raise the Python ceiling, but the structural advantage of the compiled, no-GIL tier on light endpoints holds.

Everything MobilityAPI Implements

COLLECTIONS

- ✓ create (POST)
- ✓ list (GET)
- ✓ retrieve (GET)
- ✓ replace (PUT)
- ✓ delete (DELETE)

MOVING FEATURES

- ✓ insert Feature / FeatureCollection
- ✓ list + paginate (limit)
- ✓ retrieve single
- ✓ delete
- ✓ MovingFeaturesJSON validation

QUERYING

- ✓ bbox spatial filter
- ✓ datetime instant / interval
- ✓ subTrajectory clipping
- ✓ leaf parameter

TEMPORAL GEOMETRY

- ✓ get sequence (GET)
- ✓ append primitive (POST)
- ✓ delete primitive (DELETE)

DERIVED MEASURES

- ✓ distance
- ✓ velocity
- ✓ acceleration
- ✓ → TReal series

TEMPORAL PROPERTIES

- ✓ list / add
- ✓ read / append values
- ✓ delete property
- ✓ delete one value

Thanks for Listening

Questions?



Find Out More

- **OGC API - Moving Features - Part 1: Core**
<https://docs.ogc.org/is/22-003r3/22-003r3.html>
- **MobilityAPI source**
<https://github.com/MobilityDB/MobilityAPI>
- **MobilityDB**
<https://github.com/MobilityDB/MobilityDB> · <https://mobilitydb.com>
- **PyMEOS (temporal-data layer)**
<https://github.com/MobilityDB/PyMEOS>
- **MEOS / ecosystem overview**
<https://libmeos.org>
- **AIS data — Danish Maritime Authority**
<http://aisdata.ais.dk>